



Universidad Nacional Autónoma de México



Faculty of Engineering
Electrical Engineering Division

Course: Compilers **Group:** 5
Semester: 2026-2

Report: **Parser**

Professor:
M.C. René Adrián Dávila Pérez

Student IDs:

320060829
320317264
320206968
320025561
320266083

Delivery Date:
Sunday, May 28th, 2026

Contents

1	Introduction	2
2	Theorical Framework	3
3	Development	6
3.1	Implementación	6
3.2	Gramática	6
3.2.1	Tokens terminales	6
3.2.2	Precedencia y asociatividad de operadores	7
3.2.3	Producciones de la gramática	7
3.2.4	Justificacion de las producciones	9
3.2.5	Árbol de parseo	10
3.3	Traducción Dirigida por la Sintaxis (SDT)	11
3.3.1	Reglas SDT – Estructura del programa	11
3.3.2	Reglas SDT – Declaraciones	12
3.3.3	Reglas SDT – Asignación	13
3.3.4	Reglas SDT – Expresiones aritméticas	14
3.3.5	Reglas SDT – Condiciones	15
3.3.6	Reglas SDT – Estructuras de control	16
3.3.7	Reglas SDT – Entrada/Salida	16
3.3.8	Reglas SDT – Sentencias unarias	17
3.4	Análisis semántico	17
3.4.1	Tabla de símbolos	17
3.4.2	Reglas semánticas implementadas	17
3.4.3	Funciones auxiliares de inferencia de tipos	18
3.4.4	Proceso de recorrido del AST	18
3.5	Entrada/Salida y ejecución	19
3.6	Pruebas realizadas	19
4	Results	21
5	Conclusions	21
	References	21



1 Introduction

This document contains the necessary documentation for the comprehension of the construction of a syntactic analyzer and a Syntax-Directed Translation (SDT), which extend the lexical analyzer developed in the previous phase of this project. The system processes source code written in a subset of the C language, performing syntactic analysis through a bottom-up LALR(1) parser and semantic validation by means of SDT rules applied over the resulting Abstract Syntax Tree (AST).

The lexical phase, previously implemented using the Python Pygments library, produces a token flow that serves as input to the syntactic phase. Since PLY.YACC the tool selected for the LALR(1) parser, it requires a specific lexer interface, an adapter module was developed in order to integrate the two components. The parser then constructs an AST that is later traversed by the semantic analyzer, which enforces the typing and scoping rules of the language.

We should say that, as in the previous phase, the use of the FLEX and BISON tools is not permitted. Consequently, PLY (Python Lex-Yacc) was selected as the parser generator, which imposes specific requirements on both the grammar structure and the lexer output format, leading to a significant redesign of the Context-Free Grammar defined in the previous phase.

Objectives

- Extend the lexical analyzer developed in the previous phase into a full syntactic and semantic analysis for a subset of the C language.
- Adapt the Context-Free Grammar defined in the previous phase to be compatible with PLY.YACC's LALR(1) bottom-up parsing strategy, converting right-recursive productions to left-recursive form and delegating operator precedence to PLY's precedence mechanism.
- Construct an Abstract Syntax Tree (AST) during syntactic analysis by associating semantic actions with each grammar production rule, following the Syntax-Directed Translation approach.
- Implement a symbol table with scope management to enforce semantic rules, including variable declaration before use, type compatibility in assignments, valid operands in expressions, and correct argument usage in printf and scanf.
- Provide both a terminal-based and a Streamlit graphical interface for code input, analysis execution, and result visualization, including an interactive AST graph view.



2 Theoretical Framework

Firstly, the construction of a syntactic and semantic analyzer requires a solid understanding of several theoretical concepts that form the foundation of compiler design. [1]

In this project we address the intermediate stages of the front-end of the compiler: syntactic analysis and semantic validation through Syntax-Directed Translation. These phases work together to verify that the source code provided by the user is both structurally and semantically correct before proceeding with any code generation. As in the previous phase, the validation process is applied to code written in C that uses a specific set of instructions.

2.1 Syntactic Analysis (Parsing) and LALR(1)

The syntactic analyzer is the second stage of the compiler. It receives the token flow generated by the lexical analyzer and verifies that the sequence in order to accomplish the structural rules of the language, as defined by a Context-Free Grammar (CFG). [1]

In the previous phase of this project, a Top-Down Recursive Descent Parser was implemented, which constructs the parse tree starting from the initial grammar symbol and expanding downward toward the tokens. This approach required right-recursive production rules and explicit operator precedence layering within the grammar .

The current phase replaces this approach with a Bottom-Up LALR(1) parser, as implemented by PLY.YACC. In bottom-up parsing, the parse tree is constructed from the leaves (tokens) upward to the root (start symbol) through a sequence of shift and reduce operations. LALR(1) (Look-Ahead LR with 1 token of lookahead) is a widely used variant, it combines similar parser states to make a smaller and more efficient parsing table, while still working for most real programming languages.

A big advantage is that it handles left-recursive rules (which cause problems in top-down parsers) without issues in fact, they help make parsing faster.

Also, PLY.YACC lets you directly define operator precedence and associativity, so you don't have to complicate your grammar to handle them.

2.2 Grammar Adaptation for LALR(1)

A Context-Free Grammar (CFG) is a way to describe how a language is built. It is defined by four parts: $G=(V,T,P,S)$, where V is the set of non-terminal symbols (things that can be expanded), T is the set of terminal symbols (actual tokens of the language), P is the set of production rules written like $A \rightarrow \alpha$, and S is the start symbol. [3]

In the previous phase, the grammar used right-recursive rules to avoid left recursion, since top-down parsers cannot handle left recursion. For the LALR(1) parser, these rules were changed to left-recursive form, as recommended by PLY. For example:

$$\text{stmt_list} \rightarrow \text{stmt} \mid \text{stmt stmt_list}$$

(right-recursive, previous phase)


$$\text{stmt_list} \rightarrow \text{stmt_list stmt} \mid \text{stmt}$$

(left-recursive, current phase)

Also, operator precedence (like for arithmetic and logical operations) was removed from the grammar rules and instead defined using PLY's precedence tuple. This lets you specify priority and associativity (left, right, or none) in a simpler way, making the grammar cleaner. Finally, epsilon (empty) productions were used as little as possible. When they were necessary—such as in optional parts of a `for`-loop—an explicit empty rule was created using a non-terminal called `empty`, since PLY.YACC does not have a special way to write epsilon directly. [2]

2.3 Abstract Syntax Tree (AST)

The main goal of the parser is to take a sequence of tokens and organize them into a tree structure called an Abstract Syntax Tree (AST). Unlike a full parse tree, the AST removes unnecessary details like parentheses and punctuation, keeping only the important parts that give meaning to the program. This makes it easier to understand how the code is structured and to check that it follows the correct syntax. [1]

In this project, each node of the AST contains four things: the *kind* (the type of construct, like `"assign_stmt"` or `"if_stmt"`), the *children* (a list of child nodes or values), the *lineno* (the line number in the source code, useful for errors), and the *value* (the actual text for terminal symbols). The AST is built step by step during parsing, where each grammar rule creates its corresponding node using a function called `make_node`. [6]

2.4 Syntax-Directed Translation (SDT)

Syntax-Directed Translation (SDT) is a method where grammar rules are linked with actions that give them meaning. For each rule like $A \rightarrow \alpha$, some actions are executed when the parser uses that rule. These actions usually compute values (called synthesized attributes) that move from the child nodes up to the parent node. [1]

In this project, SDT is used in two steps. First, during parsing, each grammar rule creates an AST node as its result. Then, after parsing, a separate semantic analyzer goes through the AST and checks meanings by applying rules to each node. This two-step process separates building the structure from checking its correctness.

During semantic analysis, a detailed SDT trace is kept. For each node visited, it records the step number, the type of node, the line number, the action performed, and the state of the symbol table at that moment. This makes it easier to see what the program is doing and helps with debugging. [5]

2.5 Symbol Table and Scope Management

A symbol table is a data structure used during compilation to store information about variables (identifiers) in the program. At least, it keeps the name of each variable and its type.



It is used during semantic analysis to check that variables are declared before being used and that their types are used correctly. [1]

Scope management is handled using a stack of dictionaries, where each dictionary represents a different block of code. When the parser enters a new block (like inside an `if`, `while`, or `for`), a new scope is added to the stack. When leaving the block, that scope is removed. This makes sure that variables only exist inside the block where they were declared. When searching for a variable, the system looks from the most recent scope to the oldest one, following the usual scoping rules of C.[8]

2.6 Semantic Validation Rules

Semantic analysis checks that a program makes sense, even if it is written with correct syntax. In this project, the following simple rules are applied:

- **Declaration before use:** A variable must be declared before it is used. If not, an “undeclared variable” error is reported with the line number.
- **Type compatibility in assignment:** The value assigned to a variable must match its type. Types follow this order for compatibility: `char` $\leq$ `int` $\leq$ `float` $\leq$ `double`.
- **Valid operands in expressions:** Values used in expressions must be valid types like numbers or characters. If a variable is not declared, it causes an error that affects the whole expression.
- **printf and scanf argument validation:** The number and types of arguments must match the format specifiers. For example, `%d` is for `int`, `%f` for `float`/`double`, `%c` for `char`, and `%s` for strings. If they don’t match, an error is generated.
- **Evaluable conditions:** Conditions in `if`, `while`, and `for` must be of a type that can be evaluated as true or false, such as `bool`, `int`, `float`, `double`, or `char`.

2.7 Lexer Adapter and PLY Compatibility

PLY.YACC expects the lexer to have a `token()` method that returns one token at a time, where each token has `.type` and `.value`. When there are no more tokens, it should return `None`. However, the lexer used in this project (based on Pygments) gives a list of `(type, value)` pairs, which is not directly compatible.[7]

To fix this, an Adapter pattern was used. The class `PlyLexerAdapter` takes the list of tokens, converts the type names to the uppercase format expected by PLY, and provides a `token()` method that returns tokens one by one.

This way, the difference between the lexer and the parser is handled in one place, so both parts can work correctly without needing to change their original design. [4]

3 Development

3.1 Implementación

El compilador siguió una arquitectura dividida en tres capas, donde de primeras tenemos la capa lógica, en seguida tenemos la capa de presentación que es el frontend y por último tenemos a la capa de utilidades, para esto la capa lógica se compone de seis secciones, la primera es el analizador léxico, el adaptador de tokens, las reglas gramaticales, las estructuras del AST, las reglas de validación semántica y la fase principal que integra la lógica por completo de análisis sintáctico y semántico.

El flujo sigue un pipeline se forma secuencial, donde primero, el analizador léxico tokeniza el código fuente en C utilizando Pygments como base. Después de esto, un bloque secundario transforma la salida del lexer al formato requerido por PLY (Python Lex-Yacc) y después, el parser LALR(1) de PLY analiza los tokens y construye el AST. Por último, el módulo semántico pasa el AST aplicando las reglas de validación definidas.

3.2 Gramática

Esta gramática describe un subconjunto del lenguaje C, soportando declaraciones de variables, asignaciones, estructuras de control (`if`, `while`, `for`), operaciones aritméticas, lógicas y relacionales, así como instrucciones de entrada/salida (`printf`, `scanf`). El parser construido utiliza PLY para generar un analizador LALR(1) a partir de estas producciones.

3.2.1 Tokens terminales

La gramática se basa en los siguientes conjuntos de tokens terminales y estos son generados por el analizador léxico y transformados mediante un adaptador para ser compatibles con PLY:

Table 1: Tokens terminales de la gramática

Categoría	Tokens
Tipos de dato	INT, FLOAT, DOUBLE, CHAR
Palabras reservadas	IF, WHILE, FOR, PRINTF, SCANF
Identificadores y valores	IDENTIFIER, VALUE_NUM, VALUE_CHAR, TEXT_STRING
Operadores de asignación	ASSIGN (=), ADD_ASSIGN (+=), SUB_ASSIGN (-=), DIV_ASSIGN (/=)
Operadores aritméticos	PLUS, MINUS, TIMES, DIVIDE, MOD
Operadores relacionales	EQ, NE, LT, GT, LE, GE
Operadores lógicos	AND, OR, NOT
Operadores unarios	INC (++), DEC (--)
Puntuación	SEMICOLON, COMMA, LPAREN, RPAREN, LBRACE, RBRACE
Especial	AMPERSAND (&)



3.2.2 Precedencia y asociatividad de operadores

Para resolver ambigüedades en la gramática, se definieron las siguientes reglas de precedencia que son de menor a mayor prioridad:

Table 2: Reglas de precedencia de operadores

Prioridad	Asociatividad	Operadores
1 (menor)	Izquierda	OR ()
2	Izquierda	AND (&&)
3	No asociativo	EQ, NE, LT, GT, LE, GE
4	Izquierda	PLUS, MINUS
5	Izquierda	TIMES, DIVIDE, MOD
6 (mayor)	Derecha	NOT (!)

3.2.3 Producciones de la gramática

Regresando a la gramática original tenemos lo siguiente:

Table 3: Producciones de la gramática libre de contexto

R	No terminal	Producción
1	<i>program</i>	→ <i>stmt_list</i>
2	<i>stmt_list</i>	→ <i>stmt_list stmt</i>
3		<i>stmt</i>
4	<i>stmt</i>	→ <i>decl_num</i>
5		<i>decl_char</i>
6		<i>assign_stmt</i>
7		<i>if_stmt</i>
8		<i>while_stmt</i>
9		<i>for_stmt</i>
10		<i>print_stmt</i>
11		<i>scan_stmt</i>
12		<i>unary_stmt</i>
13	<i>decl_num</i>	→ <i>type_num</i> IDENTIFIER ;
14		<i>type_num</i> IDENTIFIER = <i>math_expr</i> ;
15	<i>decl_char</i>	→ <i>type_char</i> IDENTIFIER ;
16		<i>type_char</i> IDENTIFIER = VALUE_CHAR ;
17	<i>type_num</i>	→ int float double
18	<i>type_char</i>	→ char
19	<i>assign_stmt</i>	→ IDENTIFIER <i>assign_expr_op</i> <i>math_expr</i> ;



R	No terminal	Producción
20		IDENTIFIER = VALUE_CHAR ;
21	<i>assign_expr_op</i>	→ = += -= /=
22	<i>if_stmt</i>	→ if (<i>cond</i>) { <i>stmt_list</i> }
23	<i>while_stmt</i>	→ while (<i>cond</i>) { <i>stmt_list</i> }
24	<i>for_stmt</i>	→ for (<i>for_init</i> ; <i>for_cond</i> ; <i>for_step</i>) { <i>stmt_list</i> }
25	<i>for_init</i>	→ <i>type_num</i> IDENTIFIER
26		<i>type_num</i> IDENTIFIER = <i>math_expr</i>
27		IDENTIFIER <i>assign_expr_op</i> <i>math_expr</i>
28		ϵ
29	<i>for_cond</i>	→ <i>cond</i>
30		ϵ
31	<i>for_step</i>	→ <i>inc_dec_op</i> IDENTIFIER
32		IDENTIFIER <i>inc_dec_op</i>
33		IDENTIFIER <i>assign_expr_op</i> <i>math_expr</i>
34		ϵ
35	<i>print_stmt</i>	→ printf (<i>print_args</i>) ;
36	<i>print_args</i>	→ TEXT_STRING
37		TEXT_STRING , <i>id_list</i>
38	<i>scan_stmt</i>	→ scanf (TEXT_STRING , & IDENTIFIER) ;
39	<i>id_list</i>	→ <i>id_list</i> , IDENTIFIER
40		IDENTIFIER
41	<i>unary_stmt</i>	→ <i>inc_dec_op</i> IDENTIFIER ;
42		IDENTIFIER <i>inc_dec_op</i> ;
43	<i>inc_dec_op</i>	→ ++ --
44	<i>math_expr</i>	→ <i>math_expr</i> + <i>math_expr</i>
45		<i>math_expr</i> - <i>math_expr</i>
46		<i>math_expr</i> * <i>math_expr</i>
47		<i>math_expr</i> / <i>math_expr</i>
48		<i>math_expr</i> % <i>math_expr</i>
49		IDENTIFIER
50		VALUE_NUM
51		(<i>math_expr</i>)
52	<i>cond</i>	→ <i>cond</i> <i>cond</i>
53		<i>cond</i> && <i>cond</i>
54		<i>math_expr</i> <i>rel_op</i> <i>math_expr</i>



R	No terminal	Producción
55		<i>! cond</i>
56		(<i>cond</i>)
57		<i>math_expr</i>
58		VALUE_CHAR
59	<i>rel_op</i>	→ == != < > <= >=
60	<i>empty</i>	→ ε

3.2.4 Justificación de las producciones

La gramática se compone de 25 no terminales y 60 producciones que definen el lenguaje:

Estructura del programa: La Regla 1 define que un programa consiste en una lista de sentencias, la cual se construye de forma recursiva por la izquierda mediante las Reglas 2 y 3.

Sentencias: De las Reglas de la 4 a la 12 establecen que cada sentencia puede ser una de nueve alternativas: declaración numérica, declaración de carácter, asignación, condicional **if**, ciclo **while**, ciclo **for**, impresión con **printf**, lectura con **scanf**.

Declaraciones: De las Reglas de la 13 a la 14 definen declaraciones de variables numéricas (**int**, **float**, **double**), tanto simples como con inicialización. Para la 15 y 16 hacen lo mismo pero para el tipo carácter (Regla 18), donde la inicialización solo acepta literales de carácter.

Asignaciones: La Regla 19 define la asignación con operadores simples o compuestos (Regla 21: =, +=, -=, /=) sobre la parte aritmética y la Regla 20 permite la asignación directa de letras.

Estructuras de control: Las Reglas 22 y 23 definen los bloques de control, como son el **if** y **while**, que requieren una condición entre paréntesis y un bloque que inicia y finaliza por medio de llaves. La Regla 24 define el ciclo **for** con tres partes separados por punto y coma: inicialización (Reglas 25–28), condición (Reglas 29–30) y paso (Reglas 31–34), cada uno de los cuales puede ser vacío (ε).

Entrada/Salida: La Regla 35 indica que lleva la instrucción **printf**, cuyos argumentos se especifican en las Reglas 36–37. La Regla 38 indica que lleva **scanf**, que acepta una cadena de formato y un identificador donde tiene que seguir un **&**.

Expresiones aritméticas: Las Reglas de la 44 a la 51 indican las expresiones matemáticas con cinco operadores, identificadores, constantes numéricas y agrupación con paréntesis.

Condiciones: Las Reglas de la 52 a la 58 definen las condiciones con operadores lógicos, negación, comparaciones relacionales (Regla 59), agrupación con paréntesis y la conversión de expresiones aritméticas en booleanos.

3.2.5 Árbol de parseo

Para dar un ejemplo del funcionamiento del LALR(1), se utiliza el árbol de parseo generado para la siguiente entrada:

```
x = x + 1;
```

El parser LALR(1) implementado con PLY realiza desplazamientos shift de tokens hacia la pila y reduce cuando reconoce el lado derecho de una producción en la cima de la pila.

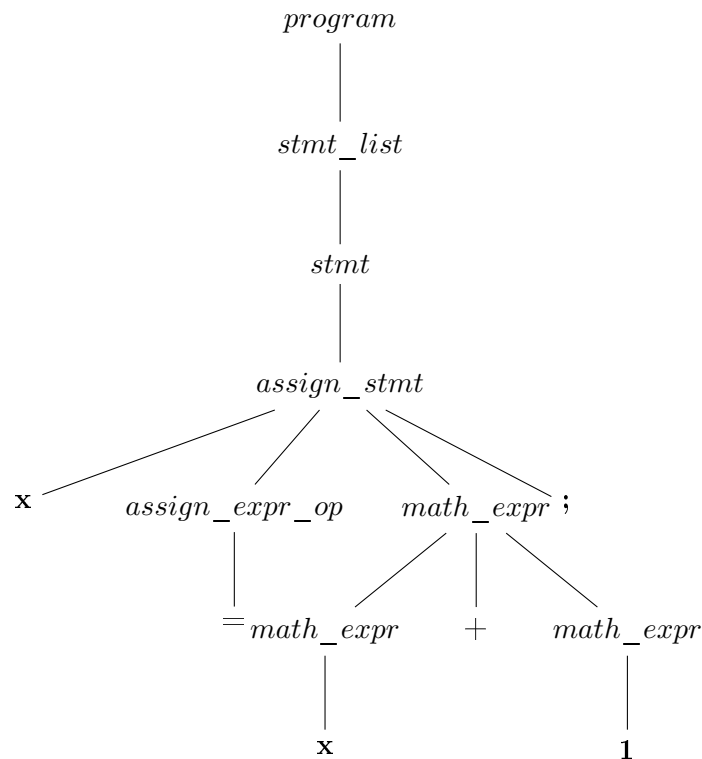


Figure 1: Árbol de parseo para la entrada `x = x + 1;`

El proceso de funcionamiento del bottom-up para esta entrada es de la siguiente manera, donde primero el analizador desplaza el token `x` hacia la pila, después desplaza el token `=` y lo reduce al no terminal haciendo uso de la Regla 21. Después desplaza el segundo token `x` y lo reduce a *math_expr* por medio de la Regla 49. Luego desplaza a la `+` y al token `1`, el cual se reduce a *math_expr* por la Regla 50.

En este punto, el parser tiene en la cima de la pila los elementos:

$$\textit{math_expr} + \textit{math_expr}$$

Que es lo mismo que la parte del lado derecho de la Regla 44.



Con esto dicho, se aplica esta reducción lo cual genera un nodo *math_expr* que representa la expresión $x + 1$. Por ultimo se desplaza el token ; y la pila contiene los elementos

IDENTIFIER *assign_expr_op math_expr* ;

Y esto es lo mismo que con la Regla 19 y se aplica esta reducción creando el nodo *assign_stmt*, el cual se reduce a *stmt* por la Regla 6, luego a *stmt_list* por la Regla 3, y finalmente al símbolo inicial *program* mediante la Regla 1.

3.3 Traducción Dirigida por la Sintaxis (SDT)

La Traducción Dirigida por el (SDT *Syntax-Directed Translation*) es que esta enlaza las reglas semánticas a cada producción de la gramática.

Para esto, con la forma de trabajo ya establecida, podemos implementar y presentar las reglas de SDT para cada producción, donde:

SymTable representa la tabla de símbolos.

.type es el atributo de tipo inferido.

.node es el nodo AST generado.

3.3.1 Reglas SDT – Estructura del programa

$$program \rightarrow stmt_list \quad \{program.node = make_node("program", stmt_list.nodes)\}$$

$$stmt_list \rightarrow stmt_list_1 stmt \quad \{stmt_list.nodes = stmt_list_1.nodes \cup \{stmt.node\}\}$$

$$stmt_list \rightarrow stmt \quad \{stmt_list.nodes = \{stmt.node\}\}$$

$$\begin{aligned} stmt \rightarrow & decl_num \mid decl_char \mid assign_stmt \mid if_stmt \\ & \mid while_stmt \mid for_stmt \mid print_stmt \mid scan_stmt \\ & \mid unary_stmt \quad \{stmt.node = X.node\} \end{aligned}$$



3.3.2 Reglas SDT – Declaraciones

```
decl_num → type_num id ;  
           {if SymTable.declare(id.name, type_num.type) then ok  
           else error("redeclaración de variable")}  
  
decl_num → type_num id = math_expr ;  
           {if SymTable.declare(id.name, type_num.type)  
           then if compatible(type_num.type, math_expr.type) then ok  
           else error("tipo incompatible en asignación")  
           else error("redeclaración de variable")}  
  
decl_char → type_char id ;  
           {if SymTable.declare(id.name, "char") then ok  
           else error("redeclaración")}  
  
decl_char → type_char id = VALUE_CHAR ;  
           {if SymTable.declare(id.name, "char") then ok  
           else error("redeclaración")}  
  
type_num → int    {type_num.type = "int"}  
type_num → float  {type_num.type = "float"}  
type_num → double {type_num.type = "double"}  
  
type_char → char {type_char.type = "char"}
```



3.3.3 Reglas SDT – Asignación

$assign_stmt \rightarrow id \ assign_op \ math_expr ;$
 {if $SymTable.lookup(id.name) = null$
 then error("variable no declarada")
 else if $assign_op.op = "="$
 then if $compatible(SymTable.lookup(id.name), math_expr.type)$
 then ok else error("tipo incompatible")
 else if $assign_op.op \in \{ "+=", "-=", "/=" \}$
 then if $numeric(SymTable.lookup(id.name)) \wedge numeric(math_expr.type)$
 then ok else error("operandos no numéricos")}

$assign_stmt \rightarrow id = VALUE_CHAR ;$
 {if $SymTable.lookup(id.name) = null$
 then error("variable no declarada")
 else if $SymTable.lookup(id.name) = "char"$ **then ok**
 else error("tipo incompatible")}

$assign_op \rightarrow = \{ assign_op.op = "=" \}$
 | $+= \{ assign_op.op = "+=" \}$
 | $- = \{ assign_op.op = "-=" \}$
 | $/ = \{ assign_op.op = "/=" \}$



3.3.4 Reglas SDT – Expresiones aritméticas

$math_expr \rightarrow math_expr_1 + math_expr_2$
 {if numeric($math_expr_1.type$) $\wedge$ numeric($math_expr_2.type$)
 then $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$
 else error("operandos no numéricos")}

$math_expr \rightarrow math_expr_1 - math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (misma regla)}

$math_expr \rightarrow math_expr_1 * math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (misma regla)}

$math_expr \rightarrow math_expr_1 / math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (misma regla)}

$math_expr \rightarrow math_expr_1 \% math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (misma regla)}

$math_expr \rightarrow \text{id}$ {if $SymTable.lookup(id.name) \neq \text{null}$
 then $math_expr.type = SymTable.lookup(id.name)$
 else error("variable no declarada")}

$math_expr \rightarrow \text{VALUE_NUM}$ { $math_expr.type = \text{infer}(\text{VALUE_NUM.value})$ }

$math_expr \rightarrow (math_expr_1)$ { $math_expr.type = math_expr_1.type$ }

3.3.5 Reglas SDT – Condiciones

$cond \rightarrow cond_1 \parallel cond_2$
 {if logical($cond_1.type$) $\wedge$ logical($cond_2.type$)
 then $cond.type = \text{"bool"}$
 else error("operandos no válidos para OR lógico")}

$cond \rightarrow cond_1 \&\& cond_2$
 {if logical($cond_1.type$) $\wedge$ logical($cond_2.type$)
 then $cond.type = \text{"bool"}$
 else error("operandos no válidos para AND lógico")}

$cond \rightarrow math_expr_1 \text{ rel_op } math_expr_2$
 {if numeric($math_expr_1.type$) $\wedge$ numeric($math_expr_2.type$)
 then $cond.type = \text{"bool"}$
 else error("operandos no numéricos en comparación")}

$cond \rightarrow ! cond_1$
 {if logical($cond_1.type$) **then** $cond.type = \text{"bool"}$
 else error("operando no válido para negación")}

$cond \rightarrow (cond_1) \quad \{cond.type = cond_1.type\}$

$cond \rightarrow math_expr \quad \{cond.type = math_expr.type\}$

$cond \rightarrow \text{VALUE_CHAR} \quad \{cond.type = \text{"char"}\}$

Donde la función logical(t) devuelve verdadero si $t \in \{\text{"bool"}, \text{"int"}, \text{"float"}, \text{"double"}, \text{"char"}\}$, son los tipos de dato que se evalúan como función lógica.

$rel_op \rightarrow == \{rel_op.op = \text{"=="}\} \mid != \{rel_op.op = \text{"!="}\}$
 $\mid < \{rel_op.op = \text{"<"}\} \mid > \{rel_op.op = \text{">"}\}$
 $\mid <= \{rel_op.op = \text{"<="}\} \mid >= \{rel_op.op = \text{">="}\}$

3.3.6 Reglas SDT – Estructuras de control

$if_stmt \rightarrow \text{if } (cond) \{ stmt_list \}$
 {if logical(*cond.type*) **then** *ok*
 else error(“condición no evaluable en if”)}

 $while_stmt \rightarrow \text{while } (cond) \{ stmt_list \}$
 {if logical(*cond.type*) **then** *ok*
 else error(“condición no evaluable en while”)}

 $for_stmt \rightarrow \text{for } (for_init ; for_cond ; for_step) \{ stmt_list \}$
 {*SymTable*.enter_scope();
 evaluar(*for_init*);
 if *for_cond* $\neq \varepsilon \wedge \neg \text{logical}(for_cond.type)$
 then error(“condición no evaluable en for”);
 evaluar(*for_step*); evaluar(*stmt_list*);
 SymTable.exit_scope() }

3.3.7 Reglas SDT – Entrada/Salida

$print_stmt \rightarrow \text{printf } (print_args) ;$
 {sea *specs* = extract_specs(**TEXT_STRING**.value)
 if |*specs*| $\neq$ |*id_list*| **then** error(“cantidad de argumentos incorrecta”)
 else $\forall (spec_i, id_i) \in \text{zip}(specs, id_list) :$
 if *SymTable*.lookup(*id_i*) = null
 then error(“variable no declarada”)
 else if $\neg \text{format_ok}(SymTable.lookup(id_i), spec_i)$
 then error(“formato incompatible”)}

 $scan_stmt \rightarrow \text{scanf } (\text{TEXT_STRING} , \&id) ;$
 {**if** *SymTable*.lookup(**id.name**) = null
 then error(“variable no declarada”)
 else sea *specs* = extract_specs(**TEXT_STRING**.value)
 if |*specs*| $\neq 1$ **then** error(“se espera exactamente un especificador”)
 else if $\neg \text{format_ok}(SymTable.lookup(id.name), specs[0])$
 then error(“formato incompatible con tipo de variable”)}



Donde la función `format_ok` verifica que sean compatibles entre un tipo de variable y un especificador de formato: `%d/%i` necesita a una variable `int`, `%f` necesita a una variable `float`, por ponerlos de ejemplo.

3.3.8 Reglas SDT – Sentencias unarias

```

unary_stmt → inc_dec_op id ;
              {if SymTable.lookup(id.name) = null
               then error("variable no declarada")
               else if ¬numeric(SymTable.lookup(id.name))
               then error("operador unario requiere tipo numérico")}

unary_stmt → id inc_dec_op ; (misma regla semántica)

inc_dec_op → ++ {inc_dec_op.op = "++"}
            | -- {inc_dec_op.op = "--"}

```

3.4 Análisis semántico

El análisis semántico es la fase siguiente del sintáctico, donde este recibe el AST generado y recorriéndolo de forma recursiva para aplicar las reglas de validación. la parte fundamental es la tabla de símbolos y unas funciones auxiliares que permiten inferir tipos, verificar compatibilidad y validar las construcciones del lenguaje.

3.4.1 Tabla de símbolos

La tabla de símbolos se implementa como una pila de diccionarios, donde cada diccionario representa un (*scope*) de variables. Cuando esta parte entra a un bloque de código como lo es el bloque de un `if`, `while` o `for`, se crea un nuevo scope apilando un diccionario vacío. Al salir del bloque, se elimina el scope superior, después las operaciones principales de la tabla son: *declare*, que registra una variable verificando que no exista redeclaración, *lookup*, que busca una variable recorriendo los scopes desde el más interno hasta el más externo, y *snapshot*, que hace que tengamos una vista combinada de todas las variables visibles en el punto actual del análisis, esto para los mensajes de error del compilador.

3.4.2 Reglas semánticas implementadas

Las reglas semánticas que el compilador valida son:

- Primero, que toda variable sea declarada antes de ser utilizada en cualquier expresión o instrucción de entrada/salida.



- Segundo, la compatibilidad de tipos en las asignaciones donde al asignar un `int` a un `float` es válido, pero no al revés.
- Tercero, que las operaciones aritméticas requieren operandos numéricos y las operaciones lógicas requieren tipos evaluables como booleanos.
- Cuarto, la validación de las instrucciones `printf` y `scanf`, donde se revusa que la cantidad de especificadores de formato sea la misma (`%d` con `int`, `%f` con `float/double`, `%c` con `char`).
- Quinto, que las condiciones utilizadas en las estructuras `if`, `while` y `for` sean expresiones que evalúen como valores booleanos de `true` o `false`.

3.4.3 Funciones auxiliares de inferencia de tipos

El parte de la semantica utiliza funciones auxiliares para la inferencia y verificación de tipos. La función de inferencia de tipo literal determina si un valor es de tipo carácter, esto lo hace por medio de ver su inicio y final, viendo si tiene comillas simples o cadena si está delimitado por comillas dobles.

La función de inferencia de tipo constante determina si un número es entero o flotante según la presencia de punto decimal.

La función de verificación numérica valida si un tipo pertenece a alguno de los tipos numéricos como lo son el (`int`, `float`, `double`, `char`).

La función de evaluabilidad lógica verifica si un tipo puede usarse como condición booleana, donde ademas acepta el tipo `bool`.

La función de compatibilidad de asignación implementa la revision de que tipo de datos aceptan otro tipo de datos: `double` acepta `float`, `int` y `char`; `float` acepta `int` y `char`; `int` acepta `char`.

La función de extracción de especificadores de formato revisa las cadenas de formato de `printf/scanf` para identificar los tipos de datos que se van tanto a leer como a imprimir (`%d`, `%f`, `%c`, `%s`) ignorando los escapes.

3.4.4 Proceso de recorrido del AST

El análisis semántico se realiza mediante dos funciones recursivas que recorren el AST.

La primera función determina el tipo de una expresión esto lo hace evaluando recursivamente los arboles generados a apartir del arbol principal, esto para identificadores, consulta la tabla de símbolos, para constantes, infiere el tipo a partir del valor, pero para operaciones binarias, verifica que ambos operandos sean válidos y aplica la promoción de tipos.

La segunda función recorre cada nodo del AST aplicando las reglas semánticas correspondientes según el tipo de nodo: para declaraciones, registra la variable en la tabla de símbolos y verifica la compatibilidad del inicializador, pero para asignaciones, verifica que la variable



exista y que el tipo sea compatible, despues para estructuras de control, revisa que la condición es correcta, ahora para la parte de instrucciones de E/S, verifica los especificadores de formato y los argumentos.

3.5 Entrada/Salida y ejecución

El programa soporta dos modos de ejecución: por línea de comandos (modo terminal) y mediante una interfaz gráfica de usuario implementada con Streamlit.

En el modo terminal, el programa presenta un menú interactivo donde el usuario puede elegir entre ingresar código C directamente por teclado o cargar un archivo desde la carpeta de entradas. El programa procesa el análisis léxico, sintáctico y semántico de manera secuencial, y muestra un reporte detallado que incluye la tabla de tokens, los tokens agrupados por tipo, el resultado del análisis sintáctico, los errores encontrados (si los hay) y la traza del análisis semántico. En caso de que se detecte un error sintáctico, el análisis semántico no se ejecuta, ya que no es posible construir un AST válido. El programa permite ejecutar múltiples análisis de forma consecutiva sin reiniciar la aplicación.

En el modo de interfaz gráfica, se ofrece la misma funcionalidad pero con una presentación más visual e interactiva. Cada fase del análisis se puede visualizar de forma organizada con colores y formatos que facilitan la lectura. Adicionalmente, la UI permite visualizar el grafo del AST generado, el cual soporta zoom para examinar árboles de gran tamaño.

3.6 Pruebas realizadas

La siguiente tabla resume todos los casos del compilador: el caso válido, los casos con errores semánticos y los casos con errores sintácticos.

Table 4: Clasificación de errores detectados por el parser y el sistema SDT.

Tipo de error	Ejemplo	Descripción
Caso válido. Programa integrado	<pre>int x = 1; float y = 2; char c = 'a'; x = x + 1; y = y + x; if (x < y && y > 0) { x += 1; } printf("%d %f %c", x, y, c); scanf("%d", &x);</pre>	Declaraciones de tres tipos distintos (Reglas 14, 16), asignaciones simples y compuestas (Regla 19), expresiones aritméticas con promoción de tipos (Regla 44), condicional con operadores lógicos y relacionales (Reglas 22, 53, 54), y E/S con múltiples especificadores de formato (Reglas 35, 38). La tabla de símbolos registra <code>x:int</code> , <code>y:float</code> , <code>c:char</code> . Resultado: Sintaxis VÁLIDA, Semántica VÁLIDA.



Tipo de error	Ejemplo	Descripción
Error sintáctico. Declaración incompleta	<pre>int x = ;</pre>	Falta la expresión de inicialización antes del punto y coma. El parser reporta: Syntax error near ';' (token SEMICOLON) at line 1. No se genera AST y el análisis semántico no se ejecuta.
Error sintáctico. Estructura de control sin paréntesis	<pre>int x = 1; if x > 0 { x = x + 1; }</pre>	La sentencia if omite los paréntesis obligatorios alrededor de la condición. El parser reporta: Syntax error near 'x' (token IDENTIFIER) at line 2. La Regla 22 requiere explícitamente los tokens LPAREN y RPAREN.
Error semántico. Variable no declarada	<pre>int x = 1; y = x + 2;</pre>	La variable y se utiliza en una asignación sin haber sido declarada previamente. La sintaxis es válida según la Regla 19, pero al recorrer el AST la tabla de símbolos no encuentra y. Se reporta: Semantic error: variable 'y' not declared at line 2.
Error semántico. Tipo incompatible en asignación	<pre>char c = 'a'; c = 25;</pre>	Se intenta asignar un valor entero (int) a una variable de tipo char. La función de compatibilidad rechaza la asignación ya que char no acepta int en la jerarquía de promoción. Se reporta: Semantic error: cannot assign 'int' to 'char' at line 2.
Error semántico. Variable no declarada en condición	<pre>int x = 1; if (z > 0 && x) { x = x + 1; }</pre>	La condición del if referencia la variable z que no fue declarada. La sintaxis es válida según la Regla 22, pero al evaluar la condición (Regla 54), la búsqueda de z en la tabla de símbolos falla. Se reporta: Semantic error: variable 'z' not declared at line 2.

Tipo de error	Ejemplo	Descripción
Error semántico. Argumentos de printf	<pre>int x = 10; printf("%d %d", x);</pre>	La cadena de formato contiene dos especificadores (%d %d), pero solo se proporciona un argumento (x). La regla SDT de printf extrae los especificadores y verifica la correspondencia. Se reporta: Semantic error: printf expected 2 args but got 1 at line 2.
Error semántico. Formato incompatible en scanf	<pre>char c = 'a'; scanf("%d", &c);</pre>	El especificador %d requiere una variable de tipo int, pero c fue declarada como char. La función de compatibilidad de formato verifica que %d solo es compatible con int. Se reporta: Semantic error: scanf format '%d' incompatible with 'char' at line 2.
Error semántico. Condición con variable no declarada	<pre>int x = 1; if (q > 0) { x = x + 1; }</pre>	La variable q no existe en la tabla de símbolos. Al evaluar la condición del if, la búsqueda falla antes de poder verificar la operación relacional. Se reporta: Semantic error: variable 'q' not declared at line 2.

Los nueve casos de prueba producen exactamente el resultado esperado. En los casos con error sintáctico, el análisis semántico no se ejecuta ya que no es posible construir un AST válido. En todos los casos semánticos, la sintaxis es válida y el error es detectado correctamente por las reglas SDT durante el recorrido del AST, generando mensajes que incluyen el tipo de error, la variable o token involucrado y el número de línea correspondiente.

4 Results

5 Conclusions

References

- [1] A. Holub, *Compiler Design in C*. Prentice-Hall, 1990, ISBN: 0-13-155045-4. [Online]. Available: <https://holub.com/goodies/compiler/compilerDesignInC.pdf>



- [2] D. Beazley. “Ply (python lex-yacc) documentation.” version 3.11. [Online]. Available: <https://www.dabeaz.com/ply/ply.html>
- [3] GeeksforGeeks. “What is context-free grammar? ”[Online]. Available: <https://www.geeksforgeeks.org/theory-of-computation/what-is-context-free-grammar/>
- [4] GeeksforGeeks. “Lalr parser (look-ahead lr). ”[Online]. Available: <https://www.geeksforgeeks.org/compiler-design/lalr-parser-with-examples/>
- [5] GeeksforGeeks. “Syntax directed translation in compiler design. ”[Online]. Available: <https://www.geeksforgeeks.org/syntax-directed-translation-in-compiler-design/>
- [6] dev.to. “Abstract syntax tree (ast) deep dive: From theory to practical compiler implementation. ”[Online]. Available: https://dev.to/min_yi_e5fbf986e24f1c42df/abstract-syntax-tree-ast-deep-dive-from-theory-to-practical-compiler-implementation-4jpo
- [7] GeeksforGeeks. “Adapter design pattern. ”[Online]. Available: <https://www.geeksforgeeks.org/adapter-pattern/>
- [8] GeeksforGeeks. “Symbol table in compiler design. ”[Online]. Available: <https://www.geeksforgeeks.org/symbol-table-compiler/>
- [9] Pygments Contributors. “Builtin tokens,” Accessed: Mar. 13, 2026. [Online]. Available: <https://pygments.org/docs/tokens/>
- [10] GeeksforGeeks. “Ambiguous grammar. ”[Online]. Available: <https://www.geeksforgeeks.org/compiler-design/ambiguous-grammar/>
- [11] C. Wong. “Cfg developer. ”[Online]. Available: <https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/>